TASKFLOW

Görev ve Proje Yönetim Sistemi REST API: Tasarım Çıktısı

Geliştirici: Barış Birdal
E-posta: barisbirdal3@gmail.com
GitHub: github.com/barisbirdal/taskflow-api
Ekim 2026 · Bitirme Projesi

1. Projenin Amacı ve Kapsamı

TaskFlow, bir yazılım şirketindeki görevlerin, projelerin ve çalışan sorumluluklarının tek bir merkezden takip edilmesini sağlayan bir REST API'dir. Sistem; görevlerin oluşturulmasını, çalışanlara atanmasını, durum ve öncelik takibini, filtrelenmesini ve temel raporlamayı destekler.

Proje Node.js ve Express.js ile geliştirilmiştir. Katmanlı bir yapıya sahiptir ve veriler JSON dosyalarında saklanır. Tüm zorunlu aşamalar (CRUD, routing, logger middleware, Postman testleri) ile isteğe bağlı dört pratik (filtreleme/arama/sayfalama, validation, raporlama, dokümantasyon) uygulanmıştır.

2. Sistem Senaryosu

- **Görev yönetimi:** Proje yöneticisi başlık, açıklama, öncelik (`low` , `medium` , `high`), son tarih ve sorumlu çalışan bilgisiyle görev oluşturur.
- **Durum takibi:** Çalışan göreve başladığında durumu `todo` → `in-progress` , bitirdiğinde `completed` olarak günceller.
- **Filtreleme ve arama:** Görevler durum, öncelik, sorumlu kişi ve projeye göre filtrelenebilir; başlık ve açıklamada büyük/küçük harf duyarsız arama yapılabilir.
- **Raporlama:** Tamamlanan ve bekleyen görev sayıları, tamamlanma oranı ve gecikmiş görevler sorgulanabilir.
- **Proje ilişkisi:** Görevler projelere bağlanabilir. Görevi bulunan bir proje silinmek istendiğinde işlem `409 Conflict` ile engellenir.

3. Sistem Mimarisi

Proje, her katmanın tek bir sorumluluğu olacak şekilde düzenlenmiştir. Bir isteğin sistem içindeki yolu:

```
İstek → express.json() → Logger → Router → Validation → Controller → Model → JSON dosyası
        Herhangi bir adımda hata oluşursa → errorHandler → standart JSON hata cevabı
```

Katman	Klasör	Sorumluluk
Routing	`src/routes/`	Gelen isteği URL ve HTTP metoduna göre ilgili controller fonksiyonuna yönlendirir.
Middleware	`src/middlewares/`	İstek loglama (logger), gövde ve id doğrulama (validation), 404 ve merkezi hata yönetimi.
Controller	`src/controllers/`	İş mantığı: varsayılan değerler, filtreleme, sıralama, sayfalama, proje varlık kontrolü.
Model	`src/models/`	Veri erişimi. CRUD fonksiyonları ortak bir fabrika (`createModel`) ile görev ve proje için üretilir.
Veri	`src/utils/fileDb.js`	JSON dosyalarını okuma/yazma. Harici veritabanı kurulumu gerektirmez.

4. Veri Modeli Tasarımı

4.1. Görev (Task)

Alan	Tip	Zorunluluk	Kural	Varsayılan
`id`	Number	Sistem	Otomatik artan tam sayı	—
`title`	String	Zorunlu	3–100 karakter	—
`description`	String	Opsiyonel	En fazla 500 karakter	`""`
`status`	String	Opsiyonel	`todo`, `in-progress`, `completed`	`todo`
`priority`	String	Opsiyonel	`low`, `medium`, `high`	`medium`
`assignee`	String / Null	Opsiyonel	2–50 karakter	`null`
`projectId`	Number / Null	Opsiyonel	Pozitif tam sayı, var olan bir proje	`null`
`dueDate`	String / Null	Opsiyonel	Tarih, `YYYY-MM-DD`	`null`
`createdAt`	String (ISO)	Sistem	Oluşturulma zamanı	Otomatik
`updatedAt`	String (ISO)	Sistem	Son güncellenme zamanı	Otomatik

4.2. Proje (Project)

Alan	Tip	Kural
`id`	Number	Otomatik artan tam sayı
`name`	String	Zorunlu, 3–100 karakter
`description`	String	Opsiyonel, en fazla 500 karakter
`createdAt` / `updatedAt`	String (ISO)	Otomatik zaman damgaları

İlişki (1:N): Bir proje birden fazla göreve sahip olabilir (`Task.projectId → Project.id`). Görevi olan bir proje silinemez (`409 Conflict`).

5. API Tasarımı ve Endpoint Listesi

Base URL: `http://localhost:3000/api`

Metot	Endpoint	Açıklama	Başarılı
POST	`/tasks`	Görev ekler	201
GET	`/tasks`	Görevleri listeler (filtre, sıralama, sayfalama)	200
GET	`/tasks/:id`	Görev detayını getirir	200
PUT	`/tasks/:id`	Görevi tamamen günceller	200
PATCH	`/tasks/:id`	Yalnızca gönderilen alanları günceller	200
DELETE	`/tasks/:id`	Görevi siler	200
GET	`/tasks/search?keyword=`	Başlık ve açıklamada arama	200
GET	`/tasks/assignee/:name`	Kişiye atanmış görevler	200
GET	`/reports/completed`	Tamamlanan görev sayısı	200

GET	`/reports/pending`	Bekleyen görev sayısı ve dağılımı	200
GET	`/reports/summary`	Genel sistem özeti	200
GET POST	`/projects`	Projeleri listeler / proje ekler	200 / 201
GET PATCH DELETE	`/projects/:id`	Proje detay / güncelle / sil	200 (409)
GET	`/projects/:id/tasks`	Projeye bağlı görevler	200

Listeleme parametreleri: `?status=`, `?priority=`, `?assignee=`, `?projectId=`, `?sort=createdAt` (azalan için `-createdAt`), `?page=1&limit=10`

6. Doğrulama ve Hata Yönetimi

Tüm cevaplar ortak bir JSON yapısındadır. Hatalı istekte uygun HTTP kodu ve açıklayıcı mesaj döner:

Kod	Durum
400	Eksik/geçersiz alan, geçersiz id, geçersiz sorgu parametresi, bozuk JSON
404	Kayıt veya endpoint bulunamadı
409	Görevi olan proje silinmeye çalışıldı

```
// POST /api/tasks  { "title": "ab", "priority": "urgent" }  →  400 Bad Request
{
  "success": false,
  "message": "Doğrulama hatası",
  "errors": [
    { "field": "title",    "message": "title en az 3 karakter olmalıdır" },
    { "field": "priority", "message": "priority şu değerlerden biri olmalıdır: low, medium, high" }
  ]
}
```

7. Logger Middleware

Her istek; zaman damgası, metot, endpoint, durum kodu ve işlem süresiyle konsola ve `logs/requests.log` dosyasına yazılır:

```
[2026-10-14T15:35:26.459Z] POST /api/tasks 201 - 14.7 ms
[2026-10-14T15:38:42.747Z] GET /api/tasks 200 - 2.3 ms
[2026-10-14T15:40:52.138Z] DELETE /api/tasks/6 200 - 3.0 ms
```